Building a Production-Style Backend API with Async Workers: An AI Data Platform Architect's View

Modern AI and data platforms are not built around a single request-response cycle. They are built around work that arrives continuously, moves through multiple stages, changes state over time, and must remain observable, recoverable, and auditable.

A user uploads a document. A system generates embeddings. A batch process transforms files. A model evaluation run takes several minutes. A report export needs to be generated. A pipeline validates incoming data quality. A downstream system waits for enrichment, classification, or inference.

None of these workloads fit cleanly into a synchronous HTTP request.

That is the architectural problem this backend pattern solves.

The project described here uses FastAPI, Redis, PostgreSQL, workers, and Docker Compose to implement a production-style asynchronous job processing system. At a surface level, it is a compact backend API. From an AI Data Platform Architect's perspective, however, it represents something more important: a foundational control-plane pattern for reliable AI and data infrastructure.

The core idea is simple:

The API should accept work quickly, persist durable state, enqueue the job, and return a job ID. The worker should process the long-running task separately. PostgreSQL should remain the source of truth. Redis should act as the queue. The client should poll for status until the job reaches a terminal state.

That pattern is small enough to understand locally, but powerful enough to show up across production AI platforms, data engineering systems, analytics automation tools, RAG pipelines, evaluation systems, and workflow orchestration layers.

Why This Pattern Matters in AI Data Platforms

AI data platforms manage workloads that are both data-intensive and time-sensitive. Some operations are fast: validating a request, authenticating a user, writing metadata, checking health, or returning job status. Other operations are slow: generating embeddings, indexing vectors, calling an LLM, parsing documents, evaluating model output, transforming datasets, or waiting on third-party services.

A strong AI data architecture separates these two categories of work.

Fast request handling belongs in the API layer. Long-running execution belongs in the worker layer. Durable state belongs in the database. Temporary coordination belongs in the queue.

This separation matters because AI systems introduce unpredictable latency. A model call may take seconds. A document indexing task may take minutes. A data quality scan may depend on volume. A batch evaluation may include hundreds or thousands of examples. If all of that work happens inside the HTTP request lifecycle, the platform becomes fragile. Requests time out, users lose visibility, retries become unsafe, and operational debugging becomes painful.

The async worker pattern solves this by converting a long-running operation into a durable job.

Instead of asking the client to wait, the platform gives the client a handle: a job ID. That job ID becomes the contract between the user-facing system and the backend processing layer.

From an architecture standpoint, this is a shift from “do the work now” to “accept the work, track the work, process the work, and expose the state of the work.”

That distinction is critical for AI systems.

The System at a Glance

The system contains four primary runtime components:

1. **FastAPI API Service**
2. **Redis Queue**
3. **Worker Service**
4. **PostgreSQL Database**

Each component has a specific responsibility.

The FastAPI service handles HTTP communication. It validates incoming requests, creates job records, pushes job IDs to Redis, and exposes endpoints for checking job status. It is the public-facing interface of the system.

Redis acts as the lightweight queue. It does not store the entire job payload. It only carries the job ID. This is an important design choice because the queue should be fast, minimal, and disposable compared to the durable system of record.

The worker service consumes jobs from Redis. When a job ID appears, the worker retrieves the full job record from PostgreSQL, marks the job as running, executes the relevant task handler, and writes the result or error back to the database.

PostgreSQL is the durable source of truth. It stores the job payload, job status, results, errors, timestamps, and any metadata needed for auditability or debugging.

This creates a clean division of responsibility:

- The API accepts and tracks work.
- Redis coordinates work.
- The worker executes work.
- PostgreSQL remembers work.

That architecture is intentionally simple, but it models the same control flow used in more advanced AI infrastructure.

The Job Lifecycle

The job lifecycle begins when a client submits a request to `POST /jobs`.

The request includes a job type and a payload. The API validates the payload using Pydantic models. This validation step matters because invalid work should be rejected before it enters the processing system. Once the request passes validation, the API creates a job record in PostgreSQL.

At that point, the job becomes durable.

This is an important production principle: a job should be persisted before it is queued. If the job is only placed in a queue but not stored durably, a system failure can cause the work to disappear. By inserting the job record into PostgreSQL first, the platform creates a stable record of the request.

After the job record is created, the API pushes the job ID into Redis using a queue operation such as `LPUSH`. The API then returns `202 Accepted` to the client.

That status code is meaningful. It tells the client: “Your request was accepted, but the work is not complete yet.”

The worker then waits on Redis using a blocking operation such as `BRPOP`. When a job ID becomes available, the worker consumes it, loads the corresponding job from PostgreSQL, updates the status to `running`, and executes the proper task handler.

When execution succeeds, the worker writes the result back to PostgreSQL and marks the job as `completed`.

When execution fails, the worker writes the error back to PostgreSQL and marks the job as `failed`.

The client can then call `GET /jobs/{job_id}` to retrieve the current state of the job. The job may be pending, queued, running, completed, or failed.

This state model gives the platform transparency. Instead of hiding work behind a black box, the system exposes where the job is in its lifecycle.

Why the State Model Is Architecturally Important

The explicit state model is one of the most important parts of this design.

A job is not just “done” or “not done.” In a reliable platform, the system needs to represent intermediate and terminal states. A simple model might include:

- `pending`
- `queued`
- `running`
- `completed`
- `failed`

Each state communicates something operationally meaningful.

`pending` means the job has been accepted or initialized but may not yet be fully handed off.

`queued` means the job is waiting for a worker.

`running` means a worker has claimed the job and is actively processing it.

`completed` means the job finished successfully and has a result.

`failed` means the job reached a terminal error state and should expose failure information.

In AI and data systems, this state model becomes even more important because jobs may involve multiple expensive or failure-prone steps. A document ingestion task may parse a file, chunk the text, generate embeddings, write vectors, and update metadata. A model evaluation job may load a dataset, run prompts, compare outputs, calculate scores, and write metrics. A reporting job may query a warehouse, generate a file, upload the artifact, and notify the user.

Each step can fail differently.

A clear job state model gives the platform a vocabulary for tracking progress, debugging errors, designing retries, and eventually building richer observability.

For an AI Data Platform Architect, this is not just an implementation detail. It is the beginning of a control plane.

Redis as the Queue, PostgreSQL as the System of Record

One of the most important architectural decisions in this system is using Redis and PostgreSQL for different purposes.

Redis is used for queue coordination. PostgreSQL is used for durable state.

That distinction prevents a common design mistake: treating the queue as the database.

Redis is excellent for fast queue operations. It can push and pop job IDs quickly, support blocking worker loops, and provide a lightweight handoff between the API and worker. However, Redis should not be the only place where job history, payloads, results, and errors live.

PostgreSQL is better suited for durable records. It supports structured querying, transactions, indexes, timestamps, audit fields, and historical analysis. It can answer questions like:

- How many jobs failed today?
- Which job types take the longest?
- Which payloads caused errors?
- How many jobs are currently running?
- When was this job created?
- When did it complete?
- What result was produced?
- What error was returned?

By storing the complete job record in PostgreSQL and only placing the job ID in Redis, the system keeps the queue lightweight while preserving durable traceability.

This is the same principle used in many production AI and data platforms. The queue is a transport mechanism. The database is the source of truth.

Why Separating the API and Worker Matters

Separating the API from the worker is a core platform architecture decision.

The API and the worker have different performance profiles. The API needs to be responsive, predictable, and available to clients. The worker needs to handle slower, heavier, and potentially unpredictable processing.

If both responsibilities live in the same runtime path, the system becomes harder to scale and harder to reason about.

When they are separated, the platform gains several advantages.

First, the API can scale independently. If request volume increases, more API replicas can be added.

Second, workers can scale independently. If processing volume increases, more worker replicas can be added.

Third, failures are isolated. If a worker crashes while processing a task, the API can still accept requests and expose status endpoints. If the API restarts, previously created jobs remain stored in PostgreSQL.

Fourth, responsibilities become clearer. The API owns validation and orchestration. The worker owns execution. PostgreSQL owns durability. Redis owns queue handoff.

This service boundary becomes especially valuable in AI systems where processing workloads evolve over time. A simple text-processing worker today may become an embedding worker tomorrow. Later, it may become a document indexing service, a model evaluation runner, or a batch inference processor.

The API contract can remain stable while the worker implementation evolves behind the scenes.

From Toy Tasks to AI Platform Workloads

The project includes simple task types such as word counting, reversing text, and simulating a slow task. These are intentionally small examples. Their purpose is not to build an advanced AI application immediately. Their purpose is to prove the infrastructure pattern.

Once the pattern works, the task handlers can be replaced with more realistic AI and data platform workloads.

For example, a `word_count` task could evolve into a document profiling job.

A `reverse_text` task could evolve into a text transformation or normalization job.

A `slow_task` could evolve into a stand-in for embedding generation, model inference, file parsing, data validation, or report generation.

In a production AI platform, the same architecture could support job types such as:

- `document_ingestion`
- `generate_embeddings`
- `index_vectors`
- `run_rag_evaluation`
- `batch_model_inference`
- `profile_dataset`
- `validate_data_quality`
- `generate_report`
- `classify_document`
- `extract_entities`
- `refresh_semantic_layer`
- `sync_external_source`

Each job type can be routed to a different handler. Over time, those handlers may become separate worker services, each optimized for a specific workload.

This is where the architecture becomes extensible. The system starts with a small task registry, but the pattern can grow into a broader job execution framework.

The AI Data Platform Architect’s Mental Model

From an AI Data Platform Architect’s perspective, this project is not just about building an API. It is about designing an execution layer.

The execution layer answers a critical question:

How does the platform reliably accept, track, process, and expose the state of long-running AI and data tasks?

That question appears across many platform capabilities.

In a RAG platform, ingestion jobs need to parse documents, chunk text, generate embeddings, and update vector indexes.

In an evaluation platform, jobs need to run datasets through prompts, compare outputs, calculate metrics, and store results.

In an analytics platform, jobs may need to refresh semantic models, generate extracts, or produce scheduled reports.

In a data engineering platform, jobs may need to validate schemas, transform files, or monitor pipeline quality.

In an agentic platform, jobs may represent tool execution, multi-step workflows, retrieval operations, or background reasoning tasks.

The architectural pattern remains the same:

- Accept the work.
- Persist the job.
- Queue the job ID.
- Process asynchronously.
- Update durable state.
- Expose the result.

That is why this backend pattern is so useful. It is small enough to build as a learning project, but it represents a core abstraction used in serious AI infrastructure.

Reliability as a First-Class Design Goal

Reliable AI infrastructure is not only about model quality. It is also about operational behavior.

Can the system recover from failure?

Can users see what happened?

Can developers debug issues?

Can jobs be retried safely?

Can the platform identify where work is stuck?

Can a failed job expose a meaningful error?

This project introduces several reliability practices that matter in production systems.

Health checks verify that the API, Redis, PostgreSQL, and worker dependencies are available. This is essential for local development, container orchestration, and eventual production deployment.

Structured logs provide machine-readable operational events. Including fields such as `job_id`, `job_type`, and `service` makes it easier to trace a request across the API, queue, worker, and database.

Explicit job states make the lifecycle observable.

Durable PostgreSQL state ensures that the platform can inspect history even after process restarts.

Docker Compose networking and dependency health conditions make the local environment closer to a real multi-service system.

These practices may seem simple, but they are the difference between a demo and a platform foundation.

Production Hardening: What Comes Next

The project already points toward several production improvements. From an AI Data Platform Architect's view, these are not optional extras. They are the natural next layer of maturity.

Retries

Many AI and data workloads fail for transient reasons. A model provider may rate limit a request. A network call may timeout. A file may be temporarily unavailable. A database connection may drop.

A production system should support controlled retries.

Retries should include limits, backoff, and error classification. Not every error should be retried. A malformed payload should fail permanently. A temporary external service failure may be retried.

Dead-Letter Queues

When jobs fail repeatedly, they should not loop forever. A dead-letter queue gives the platform a place to send jobs that cannot be processed successfully after the allowed retry attempts.

This is important for operational triage. Failed jobs can be inspected, replayed, corrected, or archived.

Job Timeouts

A worker should not allow a job to run forever. Long-running tasks need timeout boundaries. Without timeouts, a worker can become stuck and reduce processing capacity.

Timeouts are especially important for AI workloads that call external systems or process variable-sized inputs.

Idempotency Keys

Clients may retry requests. Networks may fail. Users may submit the same job more than once.

Idempotency keys help prevent duplicate work. If the same client submits the same logical request twice, the platform can return the existing job instead of creating a duplicate.

This is especially important when the job is expensive, such as embedding generation, model inference, or report generation.

Authentication and Authorization

A production job API should know who submitted the job and whether they are allowed to access the result.

This matters for enterprise AI systems where payloads may contain sensitive business data, PHI, PII, financial information, customer records, or proprietary documents.

Observability

Structured logs are a strong start, but production systems also need metrics and traces.

Useful metrics might include:

- jobs created per minute
- jobs completed per minute
- jobs failed per minute
- queue depth
- average processing time by job type

- p95 and p99 processing time
- worker error rate
- retry count
- time spent in queued state
- time spent in running state

Distributed tracing can connect API requests, queue events, worker execution, and database updates into a single operational view.

Artifact Storage

Some jobs produce larger outputs than should be stored directly in PostgreSQL. For example, report files, parsed documents, model output batches, evaluation artifacts, or generated datasets may belong in object storage.

In that architecture, PostgreSQL stores the metadata and pointer to the artifact, while object storage holds the larger file.

Workflow Expansion

Eventually, some jobs may become multi-step workflows. A document ingestion job might include parse, chunk, embed, index, and validate stages. A model evaluation job might include load dataset, run prompts, score outputs, aggregate metrics, and publish results.

At that point, the platform may introduce workflow orchestration. However, the same fundamental concepts remain: durable state, explicit status, asynchronous execution, and observable transitions.

How This Pattern Supports RAG Systems

Retrieval-Augmented Generation systems depend heavily on asynchronous processing.

When a user uploads documents, the platform usually cannot parse, chunk, embed, and index everything inside a single HTTP request. The ingestion process should be handled as a background job.

This architecture maps naturally to RAG ingestion:

1. The client uploads or references a document.
2. The API creates an ingestion job.
3. The job ID is queued.
4. A worker parses the document.
5. The worker chunks the content.
6. The worker generates embeddings.
7. The worker writes vectors to a vector database.
8. The worker updates PostgreSQL with job status and metadata.
9. The client polls for completion.

PostgreSQL can store document metadata, job status, ingestion history, chunk counts, error messages, and indexing timestamps. Redis can coordinate the queue. Workers can scale based on ingestion volume.

This gives the RAG platform operational visibility. Instead of saying “the document is processing,” the platform can show whether the document is queued, running, completed, or failed.

That visibility matters to users and operators.

How This Pattern Supports AI Evaluation Systems

AI evaluation is another strong use case.

Evaluation jobs are often long-running because they involve repeated model calls, dataset iteration, scoring logic, and result aggregation. Running these evaluations synchronously through an API request would be fragile.

With this pattern, an evaluation request becomes a durable job.

The payload might include:

- evaluation dataset ID
- model name
- prompt version
- scoring rubric
- sample size
- temperature settings
- evaluation type
- user or team ID

The worker executes the evaluation and writes the result to PostgreSQL. The result might include scores, failure counts, latency metrics, output samples, and references to larger artifacts.

This creates an auditable evaluation history, which is essential for AI governance. Teams can compare prompt versions, model versions, datasets, and scoring results over time.

For an AI Data Platform Architect, this is a key bridge between application infrastructure and model governance.

How This Pattern Supports Analytics and Reporting

The same async job architecture also applies to analytics workloads.

Some reports are too expensive to generate on demand inside a request. Some extracts need to query large datasets. Some dashboards need scheduled refreshes. Some semantic model operations need background processing.

A reporting platform can use this pattern to generate exports, refresh derived tables, run metric validation, or trigger downstream delivery.

For example:

- `generate_csv_export`
- `refresh_executive_report`
- `validate_metric_definitions`
- `sync_dashboard_metadata`
- `run_data_quality_checks`
- `publish_monthly_kpi_packet`

The user-facing application remains responsive because it only creates the job and returns a job ID. The worker handles the expensive processing. PostgreSQL stores status, timestamps, and result metadata.

This is especially useful in enterprise analytics environments where trust, lineage, and operational traceability matter.

Local Development with Docker Compose

Docker Compose is an important part of this project because it allows the full system to run locally as a multi-service environment.

Instead of running the API, Redis, PostgreSQL, and worker manually, Docker Compose defines the services, networking, dependencies, and health checks in one place.

This creates a better development experience and a stronger architecture learning environment.

A developer can run the system locally and observe how the components interact:

- The API starts and exposes endpoints.
- PostgreSQL stores durable job records.
- Redis handles queue operations.
- The worker consumes jobs.
- Logs show the lifecycle across services.
- Health checks confirm dependencies are available.

For an AI Data Platform Architect, this matters because platform design is not only about code. It is also about runtime topology. Docker Compose makes the architecture visible.

It teaches that a backend platform is a set of cooperating services, not a single script.

The Control Plane Concept

This project can be understood as the early version of a control plane.

A control plane manages the lifecycle, metadata, and state of work. It does not necessarily do all the heavy work itself. Instead, it coordinates work across services.

In this system, the control plane includes:

- job creation
- job validation
- job state transitions
- queue handoff
- status endpoints
- result retrieval
- error visibility
- operational health

As the platform matures, the control plane could expand to include:

- user ownership
- team permissions
- job priorities
- retry policies
- worker pools
- job scheduling
- audit logs
- artifact references
- cost tracking
- model version tracking
- dataset lineage
- SLA monitoring

This is how a small backend project can evolve into a serious AI infrastructure component.

Architectural Lessons

The most important lesson from this project is that asynchronous systems are about managing time and failure.

Time matters because not all work finishes quickly.

Failure matters because not all work succeeds.

A synchronous API hides both problems until they become painful. An asynchronous job system makes them explicit.

The job ID gives the client a durable handle.

The job state gives the platform visibility.

The queue decouples request handling from execution.

The database preserves history.

The worker isolates long-running processing.

The logs and health checks make the system observable.

Together, these pieces form a reliable foundation for AI and data workloads.

Final Takeaway

This FastAPI, Redis, PostgreSQL, and worker architecture is more than a backend exercise. It is a practical foundation for building AI data platforms that need to process long-running work reliably.

The architecture demonstrates several platform-level principles:

- keep APIs fast
- persist state before processing
- use queues for decoupling
- separate orchestration from execution
- make job states explicit
- store results and errors durably
- expose operational surfaces
- design for retries, failures, and future scale

For AI Data Platform Architects, this pattern is essential. Before building advanced RAG systems, model evaluation pipelines, autonomous analytics agents, or production AI workflows, teams need to understand how durable asynchronous job processing works.

The platform must be able to accept work, track work, process work, recover from failure, and expose the result.

That is the foundation this project builds.